

Assignment 2 Report – API Service, DB Service, Logging Service

SID: 12210360

Name: 黄奕凡 (Huang Yifan)

1. Implementation Procedures

This assignment consists of three major components: API Service, DB Service, and Logging Service. Below I describe the procedures used to set up the environment, generate code, and implement business logic.

1.1 Environment Setup

- **Programming Languages:** Python 3.10 for API Service, DB Service, and Logging Service.
- **Frameworks:**
 - API Service: FastAPI + Uvicorn
 - gRPC Services: Python gRPC + protobuf
- **Databases & Middleware:**
 - PostgreSQL (pre-configured)
 - Kafka (pre-configured)
- **Tools Used:**
 - `protoc` for code generation
 - `docker compose` for multi-service deployment
 - `psycopg2` for DB connections
 - `kafka-python` for Kafka producer

I first started the base containers:

```
docker compose up -d postgres kafka zookeeper
```

To verify that the database was initialized correctly:

```
make how-to-psql
```

Inside the database shell:

```
\dt
```

Expected output:

```
goodsstore=# \dt
           List of relations
 Schema |   Name   | Type  | Owner
-----+-----+-----+-----
 public | orders   | table | test
 public | products | table | test
 public | users    | table | test
(3 rows)
```

```
docker compose logs api_service --tail=30
```

1.2 Code Generation

I created two `.proto` files:

db.proto

Contains:

- ProductService
- UserService
- OrderService
- All message definitions (ProductInfo, UserInfo, OrderInfo, etc.)

logging.proto

Contains:

- LoggingService
- Client-side streaming RPC
- LogMessage, LogAck messages
-

写好proto, 它应该包含:

- ProductService
- UserService
- OrderService
- 所有 message (Product / User / Order / RegisterRequest / NewOrder 等)

Generating Python gRPC code

Inside `src/db_service`:

```
uv venv
source .venv/bin/activate
uv pip install -r requirements.txt
```

Generate code:

```
python -m grpc_tools.protoc \
  -I./proto \
  --python_out=. \
  --grpc_python_out=. ./proto/db.proto
```

Generated files:

```
db_pb2.py
db_pb2_grpc.py
```

Generate logging gRPC code

```
python -m grpc_tools.protoc \
  -I./logging_service/proto \
  --python_out=./logging_service/grpc_generated \
  --grpc_python_out=./logging_service/grpc_generated \
  ./logging_service/proto/logging.proto
```

Kafka topic verification:

```
docker exec -it kafka kafka-topics --list --bootstrap-server kafka:9092
```

1.3 API Service (REST)

API Service

- Implemented with FastAPI and follows the OpenAPI YAML specification.
- Each REST endpoint calls the DB Service using gRPC.
- JWT-based authentication is added for user-related operations.

Feature	URL	Method
Greeting	/	GET
List Products	/products	GET
Get Product	/products/{id}	GET
Register	/users/register	POST
Deactivate	/users/{user_id}/deactivate	POST
Login	/users/login	POST
Get User	/users/{id}	GET
Update User	/users/me	PUT
Place Order	/orders	POST
Cancel Order	/orders/{id}/cancel	POST

Feature	URL	Method
Get Order	/orders/{id}	GET

Each API was tested through Swagger UI (<http://localhost:18080/docs>), and screenshot evidence was collected.

1. Greeting API

GET /

return:

```
{ "message": "Welcome to SUSTech Merch Store" }
```

GET / Greeting

Parameters

No parameters

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:18080/' \
  -H 'accept: application/json'
```

Request URL

```
http://localhost:18080/
```

Server response

Code Details

200

Response body

```
{
  "message": "Welcome to SUSTech Merch Store"
}
```

Response headers

```
content-length: 44
content-type: application/json
date: Sun, 23 Nov 2020 14:20:11 GMT
server: uicorn
```

2. list-products

GET /products

return all products in database in json

GET /products List Products

Parameters Cancel

No parameters

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:18080/products' \
  -H 'accept: application/json'
```

Request URL

http://localhost:18080/products

Server response

Code	Details
200	Response body <pre>[{ "id": 1, "name": "SUSTech Hoodie", "category": "Apparel", "price": 49.99, "stock": 500 }, { "id": 2, "name": "SUSTech Water Bottle", "category": "Drinkware", "price": 19.98, "stock": 500 }, { "id": 3, "name": "SUSTech Notebook", "category": "Stationery", "price": 9.97, "stock": 500 }]</pre> Download

3. get-product

GET /products/{id}

GET /products/{product_id} Get Product

Parameters Cancel

Name	Description
product_id required	1
integer (path)	

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:18080/products/1' \
  -H 'accept: application/json'
```

Request URL

http://localhost:18080/products/1

Server response

Code	Details
200	Response body <pre>{ "id": 1, "name": "SUSTech Hoodie", "category": "Apparel", "price": 49.99, "stock": 500 }</pre> Download
	Response headers <pre>content-length: 79 content-type: application/json date: Sun, 23 Nov 2025 14:30:35 GMT server: wicora</pre>

id do not exist → 404

GET /products/{product_id} Get Product

Parameters Cancel

Name	Description
product_id * required integer (path)	10

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:18080/products/10' \
  -H 'accept: application/json'
```

Request URL

```
http://localhost:18080/products/10
```

Server response

Code	Details
404 <i>Not Found</i>	Error: Not Found Response body <pre>{ "detail": "Product not found" }</pre> Response headers <pre>content-length: 30 content-type: application/json date: Sun, 23 Nov 2025 14:31:39 GMT server: uvicorn</pre>

• 4. Register

POST /users/register

Authentication: **Not required**

Request body:

```
{ "username": "...", "password": "..." }
```

Logic:

- Hash the password using **bcrypt**.
- Invoke the DB Service's **CreateUser** RPC to create the new user.
- Write a log entry: *"Registered user X"*.
- Return the created user info (excluding any `password_hash` field).

5. Login

POST /users/login

Authentication: **Not required**

Request body:

```
{ "username": "...", "password": "..." }
```

Logic:

- Look up the user by username
(if the DB Service does not provide `GetUserByName`, the API iterates all user IDs to locate the matching username).
- Validate the password using **bcrypt**.
- On success, generate a **JWT** containing the user ID.

- Return:

```
{ "token": "<JWT token>" }
```

- Record the login action in the Logging Service.
-

6. Get User

GET `/users/{user_id}`

Authentication: **Required**

Logic:

- Decode the JWT to obtain the authenticated user ID.
 - Call the DB Service's **GetUser** RPC.
 - Return the user's basic information (excluding the password hash).
-

7. Update User

PUT `/users/me`

Authentication: **Required**

Possible request body:

```
{
  "username": "...",
  "active": true or false
}
```

Logic:

- The user may only update their **own** information.
 - Invoke the DB Service's **UpdateUser** RPC with the fields to be updated.
 - Return the updated user information (excluding sensitive fields).
 - Produce a corresponding log entry.
-

8. Deactivate User

POST `/users/{user_id}/deactivate`

Authentication: **Required**

Logic:

- Set the target user's `active` flag to `false`.
 - Call the DB Service's **UpdateUser** RPC.
 - Return the updated user information.
 - Log the deactivation event.
-

9. Place Order

POST /orders

Request body:

```
{ "product_id": x, "quantity": y }
```

Rules and logic:

- `quantity` must be ≤ 3 (as required by the assignment).
 - The DB Service computes `total_price = price × quantity`.
 - Call the DB Service's **CreateOrder** RPC.
 - Return the created order information.
 - Log the order placement.
-

10. Cancel Order

POST /orders/{id}/cancel **

Authentication: Required**

Logic:

- Invoke the DB Service's **CancelOrder** RPC.
 - Return the updated order state.
 - Write a log entry indicating that the order was canceled.
-

11. Get Order

GET /orders/{id}

Authentication: **Required**

Logic:

- Call the DB Service's **GetOrder** RPC.
- Return the detailed order information.
- Log the retrieval action.

1.4 DB Service (gRPC)

The DB service contains three CRUD RPC modules:

Products

- CreateProduct
- UpdateProduct
- ListProducts
- GetProduct

Users

- CreateUser (register)
- UpdateUser
- GetUser
- DeactivateUser

Orders

- CreateOrder
- CancelOrder
- GetOrder

Database interactions use a PostgreSQL connection pool for efficiency.

1.5 Logging Service (gRPC client-side streaming)

Implements one RPC:

```
rpc streamLogs(stream LogMessage) returns (LogAck);
```

Pipeline:

1. API Service opens a gRPC stream.
2. For each API operation, a `LogMessage` is pushed to the stream.
3. Logging Service receives the messages continuously.
4. Each message is forwarded into Kafka topic `log-channel`.
5. When the client closes the stream, the server returns `LogAck`.

Tested Kafka logs using:

```
docker exec -it kafka kafka-console-consumer \  
  --topic log-channel --bootstrap-server kafka:9092
```

1.6 OpenAPI Generation

FastAPI was used to export the automatic OpenAPI YAML:

```
docker compose exec api_service python export_openapi.py
```

Then copy to local directory:

```
docker cp assignment2-api_service-1:/app/openapi.yaml \  
  ./src/api_service/openapi.yaml
```

1.7 Docker Compose Configuration

All services are defined inside `docker-compose.yml`.

Key design points:

- All containers share a **single Docker network**.
- Services communicate via service names:
 - API → DB: `db_service:50051`
 - API → Logging: `logging_service:50052`
 - Logging → Kafka: `kafka:9092`
- `depends_on` ensures correct startup order.

2. Authentication for API Service

2.1 APIs Requiring Authentication

The following REST APIs require a valid JWT:

API	Authentication Required?
<code>POST /login</code>	No
<code>POST /register</code>	No
<code>GET /users/{id}</code>	Yes
<code>PATCH /users/{id}</code>	Yes
<code>POST /deactivate-user</code>	Yes
<code>POST /place-order</code>	Yes
<code>POST /cancel-order</code>	Yes
<code>GET /order/{id}</code>	Yes

2.2 Authentication Logic

- After login, API Service generates a JWT:
 - `user_id`
 - `role`
 - expiration time
- JWT is signed using HS256 and stored in `Authorization: Bearer <token>`
- A FastAPI dependency validates JWT, extracts `user_id`, and injects it into the endpoint handler.

Pseudo-code:

```
def get_current_user(token: str = Depends(oauth_scheme)):
    try:
        payload = jwt.decode(token, SECRET, algorithms=["HS256"])
        return payload["user_id"]
    except:
        raise HTTPException(401, "Invalid token")
```

3. Field Data Type Selection

I ensured consistent data types across:

- OpenAPI YAML
- `.proto` definitions
- PostgreSQL schema

Example: Product Price

Component	Field Type
PostgreSQL	<code>NUMERIC(10,2)</code>
Proto	<code>double price = 2;</code>
OpenAPI	<code>type: number</code>

Reason: "Price" is numeric but may include decimals, so double/NUMERIC is appropriate.

Another example for `user_id`:

- PostgreSQL: `SERIAL` or `BIGINT`
- Proto: `int64`
- OpenAPI: `type: integer, format: int64`

4. gRPC Message Encoding Analysis

I selected this message from `db.proto`:

```
message ProductInfo {
    int64 id = 1;
    string name = 2;
    double price = 3;
}
```

Example instance:

```
id = 10
name = "Apple"
price = 3.5
```

Binary Encoding Explanation

- Field 1: `id = 10`
 - `key = (1 << 3) | 0 = 0x08`
 - `value = varint(10) = 0x0A`
- Field 2: `"Apple"`
 - `key = (2 << 3) | 2 = 0x12`
 - `length = 5 = 0x05`
 - `bytes = ASCII of "Apple" = 41 70 70 6C 65`
- Field 3: `price = 3.5`
 - `key = (3 << 3) | 1 = 0x19`
 - `double(3.5) = 0x00 00 00 00 00 00 0C 40`

Full Encoded Binary

```
08 0A 12 05 41 70 70 6C 65 19 00 00 00 00 00 00 0C 40
```

Verified with Python

```
msg = ProductInfo(id=10, name="Apple", price=3.5)
print(msg.SerializeToString())
```

The output matches the manual encoding.

5. Client-Side Streaming RPC in Logging Service

The RPC signature:

```
rpc streamLogs(stream LogMessage) returns (LogAck);
```

How It Works

1. The API Service opens a gRPC stream.
2. For each event, it sends a `LogMessage` through `request_stream.send(message)`.
3. The server reads messages one-by-one using:

```
for message in request_iterator:
    kafka_producer.send("logs", message.text.encode())
```

4. After the client closes the stream, the server replies with a single `LogAck`.

Features:

- Efficient for high-frequency logs
- Reduces network overhead

- Kafka ensures persistence

6. Docker & Docker Compose Configuration

Key Concepts

- All services run in one Docker network
- Services reference each other by container name
Example: DB Service connects to PostgreSQL using host `postgres`.

docker-compose.yaml Highlights

```
services:
  api:
    build: ./api
    ports: ["8080:8080"]
    depends_on: [db-service, logging-service]

  db-service:
    build: ./db_service
    depends_on: [postgres]

  logging-service:
    build: ./logging

  postgres:
    image: postgres:14
    environment:
      POSTGRES_PASSWORD: 123456

  kafka:
    image: confluentinc/cp-kafka
```

Networking:

- Docker Compose automatically creates a shared network
- DNS resolution uses service names

7. Experiment Procedure

API Service Testing

Tools used:

- **Swagger UI** (auto-generated by FastAPI)

access <http://localhost:18080/docs> and test all api

Kafka Log Monitoring

I used the official Kafka console consumer:

```
docker exec -it kafka kafka-console-consumer \  
--topic logs --bootstrap-server localhost:9092
```

This allowed me to see real-time log entries pushed by the Logging Service.

8. gRPC Error Handling

To satisfy the general requirement on error handling, the DB Service and Logging Service implement structured gRPC error responses using standard status codes and descriptive detail messages. The service methods validate all incoming requests and return appropriate gRPC exceptions when invalid conditions are detected.

Specifically:

- **NOT_FOUND** is returned when a requested resource (e.g., user, product, order) does not exist.
- **INVALID_ARGUMENT** is used when the client sends malformed or logically incorrect input (e.g., negative quantity, missing required fields).
- **ALREADY_EXISTS** is raised when attempting to create a user with a duplicate username.
- **FAILED_PRECONDITION** is used for business-logic violations, such as cancelling an order that is already cancelled.
- **UNAUTHENTICATED** or **PERMISSION_DENIED** is triggered when JWT verification fails or when a user attempts to access resources that do not belong to them.

Conclusion

This assignment required implementing a complete microservice architecture with REST + gRPC + PostgreSQL + Kafka. I created:

- A fully functional API Service with JWT
- A DB Service with complete CRUD operations
- A Logging Service using client-side streaming to Kafka
- Dockerized deployment and testing workflow

All components were integrated successfully and tested through both API and Kafka logs.